

Classification

Simple Distance-Based Classifier

- Given: Labelled training data $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$ from two classes.
- Goal: Learn a model to predict label (class) y for a new test example $\mathbf{x}$



Simple Distance-Based Classifier

- Assume green is positive and red is negative class
- Let's say n_+ samples from positive class, n_- samples from negative class



- A simple way to assign a class is to find the distance from the mean
- This idea can be generalized to more than 2 classes as well

Distance from Means

$$\boldsymbol{\mu}_- = \frac{1}{n_-} \sum_{y_i=-1} \mathbf{x}_i \text{ and } \boldsymbol{\mu}_+ = \frac{1}{n_+} \sum_{y_i=+1} \mathbf{x}_i$$

- Once, we have the two means the training data can be discarded

$$\|\boldsymbol{\mu}_- - \mathbf{x}\|_2^2 = \|\boldsymbol{\mu}_-\|_2^2 + \|\mathbf{x}\|_2^2 - 2\boldsymbol{\mu}_-^T \mathbf{x}$$

$$\|\boldsymbol{\mu}_+ - \mathbf{x}\|_2^2 = \|\boldsymbol{\mu}_+\|_2^2 + \|\mathbf{x}\|_2^2 - 2\boldsymbol{\mu}_+^T \mathbf{x}$$

The Decision Rule

$$\begin{aligned} \mu_-^T \mu_- - 2\mu_-^T x + x^T x \\ - \mu_+^T \mu_+ + 2\mu_+^T x - x^T x \\ = \underbrace{2(\mu_+ - \mu_-)}_w^T x + b \end{aligned}$$

$$b: \|\mu_-\|_2^2 - \|\mu_+\|_2^2$$

- Let $f(x)$ be the decision rule
- $f(x) := \|\mu_- - x\|_2^2 - \|\mu_+ - x\|_2^2 = 2(\underbrace{\mu_+^T x - \mu_-^T x}_{\text{red star}}) + \|\mu_-\|_2^2 - \|\mu_+\|_2^2$

- If $f(x) > 0$ then class is +1, otherwise class is -1.

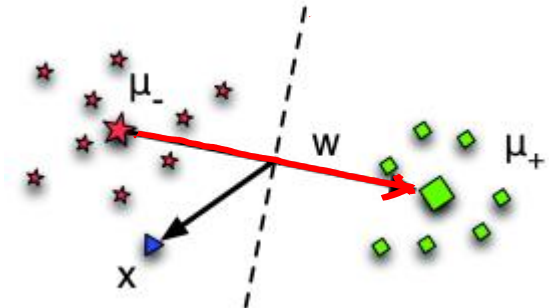
$$y = \text{sign}[f(x)]$$

$$w^T x + b = 0$$

Decision boundary

- $f(x)$ effectively denotes a hyperplane based classification rule with the vector $w = \mu_+ - \mu_-$ representing the direction normal to the hyperplane

$$\begin{aligned} x_a \quad x_b \\ w^T x_a + b = 0 \quad \& \quad w^T x_b + b = 0 \\ w^T (x_a - x_b) = 0 \end{aligned}$$

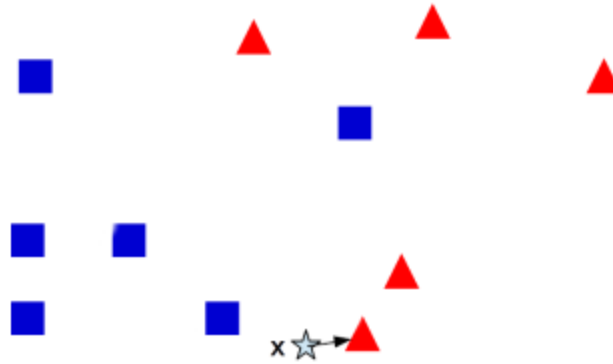


Some features of “Distance from Means”

- Simple to understand and implement
- Would usually require plenty of training data for each class (to estimate the mean of each class reliably)
- Can only learn linear decision boundaries, unless Euclidean distance is replaced by a nonlinear distance function
- Can be made more rigorous by modeling each class by a more sophisticated class probability distribution (e.g., a multivariate Gaussian) and computing distances of test points from these class distributions (rather than means)
- Several methods have this flavor, e.g., Linear (Fisher) Discriminant Analysis

Nearest Neighbor (NN) Methods

- Distance-based supervised learning method



- The label y for $x \in \mathbb{R}^d$ will be the label of its nearest neighbour in training data (a.k.a 1-NN)
- Euclidean distance can be used to find the nearest neighbor

Some Features about Nearest Neighbor

- A simple yet very effective method in practice (if given lots of training data)
 - *Provably* has an error-rate that is no worse than twice of the “Bayes optimal” classifier which assumes knowledge of the true data distribution
- Also called a memory-based or instance-based or non-parametric method
- No “model” is learned here. Prediction step uses all the training data
- Requires lots of storage (need to keep all the training data at test time)
- Prediction can be slow at test time
 - For each test point, need to compute its distance from all the training points
 - Clever data-structures or data-summarization techniques can provide speed-ups
- Need to be careful in choosing the distance function to compute distances (especially when the data dimension D is very large)
- The 1-NN can suffer if data contains outliers, or if amount of training data is small. Using more neighbors ($K > 1$) is usually more robust

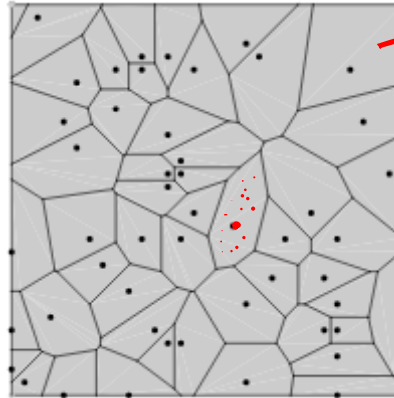
1-NN Visualization

- Every training point is a prototype.
- A test point x is assigned the label of the closest prototype
 $\hat{y}(x) = y_i$ s.t. $i = \arg \min_i \|x - x_i\|_2^2$

- 1-NN induces a Voronoi diagram (Dirichlet tessellation).

Given a set of training points
 $\{x_1, \dots, x_n\}$
Voronoi cell of x_i

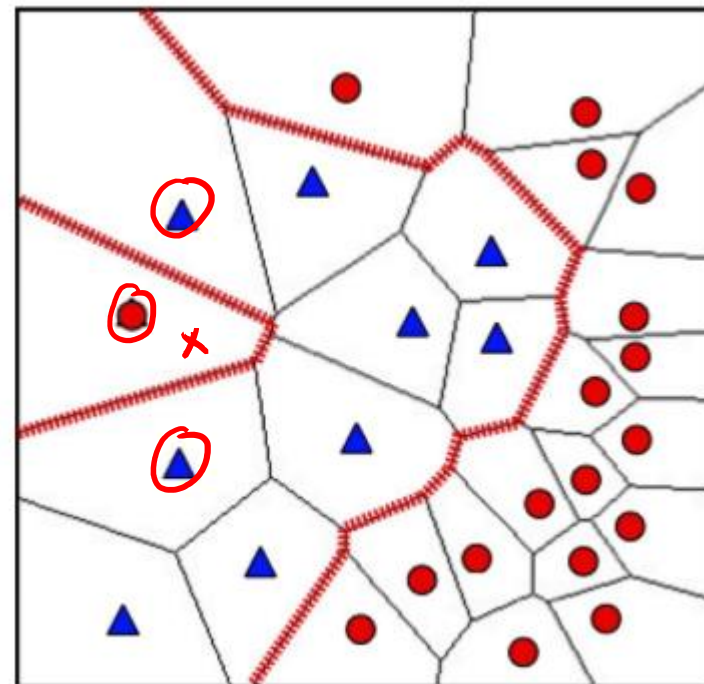
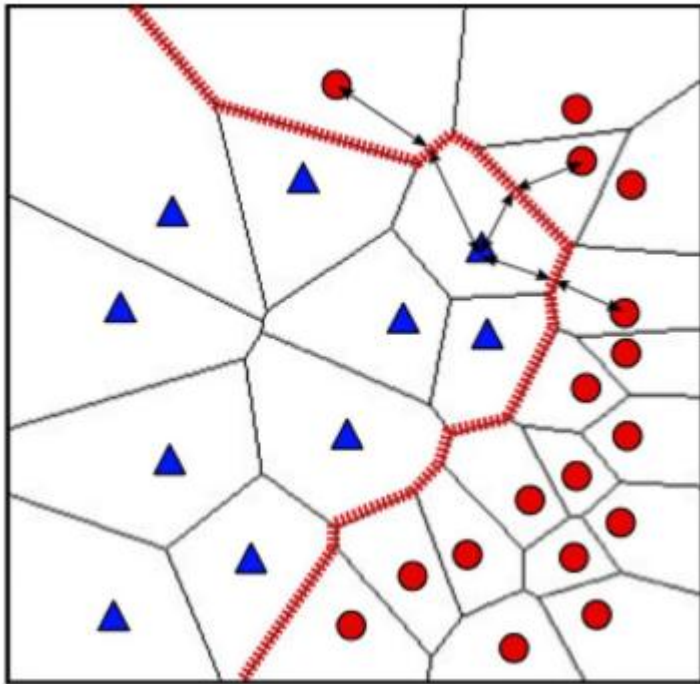
$$V_i = \left\{ x : \|x - x_i\| \leq \|x - x_j\| \forall j \neq i \right\}$$



- The partitioning of plane with n points into convex polygons
- Each polygon contains exactly one seed point
- Every point in a given polygon is closer to its seed point than to any others

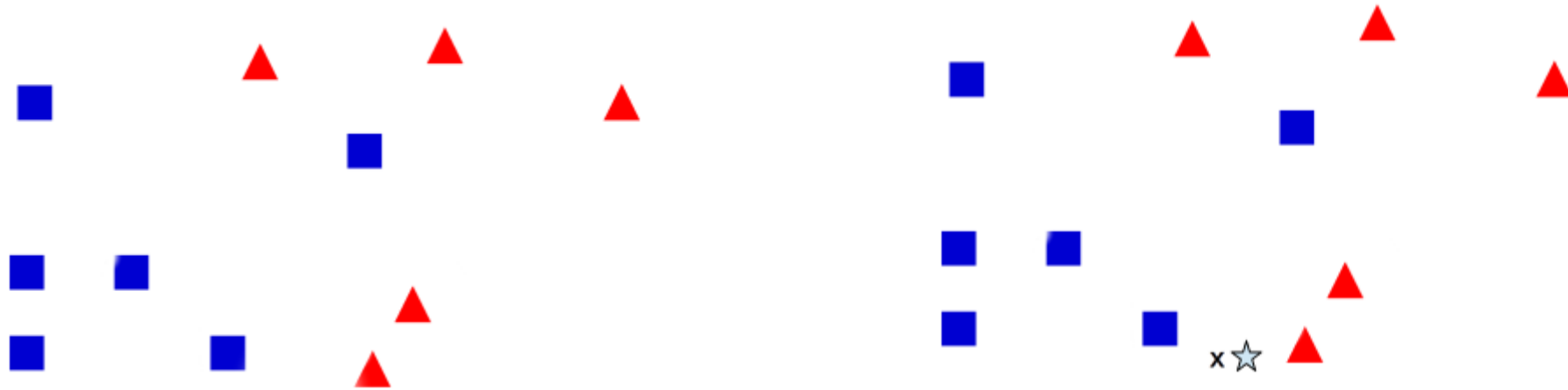
The Decision Boundary of 1-NN

- The decision boundary is composed of hyperplanes that form perpendicular bisectors of pairs of points from different classes



K - Nearest Neighbors (K -NN)

- Makes one-nearest-neighbor more robust by using more than one neighbor
- The K -NN prediction rule: Take a majority vote (or average) of the labels of $K > 1$ neighbors in the training data



K-NN Classification:-

Given training examples $\{x_i, y_i\}$

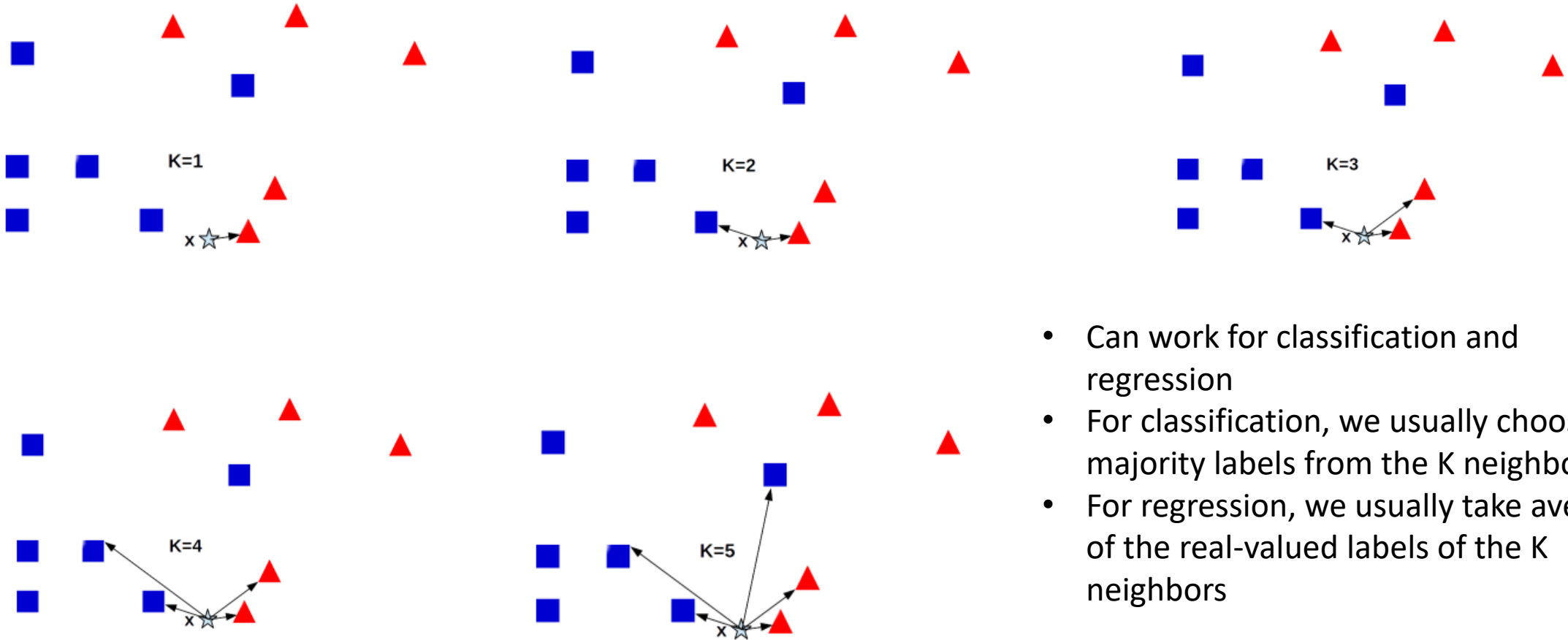
- x_i : features/examples
- y_i : label

test point x that we want to class/f.

Algorithm:-

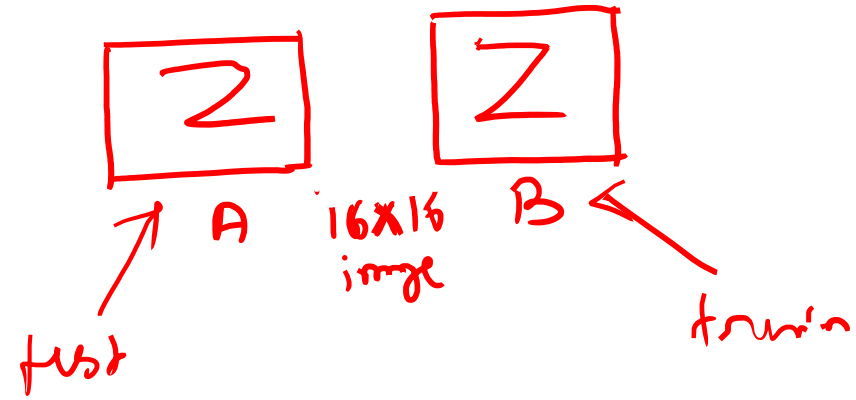
- Compute distance $D(x, x_i)$ to every training example x_i .
- Select k -closest instances $x_{i_1}, \dots, x_{i_k}$ and the labels $y_{i_1}, \dots, y_{i_k}$.
- Output the class y^* which is most frequent in $y_{i_1}, \dots, y_{i_k}$.

K - Nearest Neighbors (K -NN)



- Can work for classification and regression
- For classification, we usually choose the majority labels from the K neighbors
- For regression, we usually take average of the real-valued labels of the K neighbors

Example:- Handwritten Digit classification (mnist)



$$D(A, B) = \sqrt{\sum_{n=1}^{16} \sum_{c=1}^{16} (A_{n,c} - B_{n,c})^2}$$

FNN: ~ 95.2 %

SVM: ~ 95.8 %

Human: ~ 97.5 %